

ASE485 Capstone Project

SmartSpend — AI-Powered Personal Finance Assistant

- By: Joshua Day

Project Metrics

- Burndown rate: 19/19 features (100%)
- 20/20 Requirements
- 2/2 Sprints completed
- ~6,500+ lines of code
- 18 backend endpoints, 11 Flutter screens, 25+ test files

Problem Domain

70% of Americans live paycheck-to-paycheck, and existing budgeting tools track numbers without helping people change behavior.

- **Manual overhead** - constant categorizing and data entry
- **Generic advice** - doesn't adapt to individual spending
- **Hidden overspending** - users blow budgets without noticing
- **No starting point** - users don't know how to build a budget

Solution Domain

SmartSpend is a mobile app that learns from your real spending and gives back personalized guidance.

- **Auto-categorization** - predicts a category from transaction text
- **Budget generation** - builds budgets from your spending history
- **Savings recommendations** - personalized tips on where to cut back
- **Budget alerts** - warns you before you overspend

What Went Wrong

- Connecting the app to the backend took trial and error
- Keeping users logged in across restarts
- Building enough training examples for the ML model
- Keeping the project lightweight after adding ML libraries

What Went Well

- Sprint 1 architecture made Sprint 2 ML integration smooth
- Provider pattern stayed clean across all 11 screens
- Test coverage held strong (25+ Flutter test files + pytest)

Week-by-Week Progress Summary

- Weeks 4–5: Setup, Docker, PostgreSQL, JWT auth
- Weeks 6–7: Transactions, budgets, goals, analytics screens
- Week 8: Sprint 1 Presentation
- Weeks 9–10: Persistent auth, full live API wiring
- Weeks 11–12: ML categorization, budget generation, recommendations
- Weeks 13–14: Budget alerts, pytest suite, polish
- Week 15: Final Presentation

What I Learned

- ML models are only as good as their training data
- Clean architecture early on pays off later
- Auth and state management are harder than they look
- Testing builds confidence when refactoring
- Scope creep is real — prioritize ruthlessly

How It All Fits Together

- **Flutter** handles the UI and state (Provider pattern)
- **FastAPI** serves as the REST backend with JWT auth
- **PostgreSQL** stores users, transactions, budgets, and goals
- **scikit-learn** powers the ML categorization model
- Everything runs in **Docker** for consistent local dev

Thank You

Questions?

- Joshua Day — dayj16@mymail.nku.edu
- Repository: github.com/Jolteer/ase485-capstone-finance-ml